PROJECT REPORT

Reverse-engineering 30 years of insurance logic from data alone

Ergo turns a legacy export into exact, auditable Python. Static analysis links the tables, clustering and optimal decision trees find the rules, and an exact scorer keeps only what reproduces every value to the cent. A language model, proprietary or self-hosted, is called only where the data cannot speak for itself.

Average accuracy, public leaderboard
97.62 %
exact rows: to the cent, to the day

Perfect tasks
5 of 7
t0–t4: 100 % of 101,259 rows

Maintainability index
100.0
radon, the challenge metric

Pipeline for every task
1
7 generic steps, no per-task tooling

01 The problem

NeuralWave Life, a (fictional) life insurer in Ticino, has sold unit-linked policies since 1994. For more than 30 years a system called **Neuron** calculated every premium, fee, fund unit and payout. It began as a fork of a COBOL system, was partly rewritten in Java as Neuron NG in August 2007, and has been patched ever since; its parameters changed many times and some values were corrected by hand.

The company is migrating to Lifeware, and the new system must calculate **exactly** the same values as Neuron, for every policy, to the cent. Nobody knows how Neuron calculates them: the documentation is incomplete, the developers are gone and there is **no source code**. What is left is the data, and the result must be code that a legal auditor can check.

What survives: 27 CSV tables, 8.3 M rows

NRPOLH	NRPREM	NRBOOK	NRRISK	NRPERS
NRBROK	NRALLO	NREVNT	NRHOLD	NRADDR
NRAGRE	NREXCP	NRPROD	NRCHRG	NRCOMR
NRENTC	NRILLU	NRSURR	NRWAIV	NRTAXB
NRTAXR	NRFUND	NRFPRC	NRFXRT	NRMORT
NRDICT	NRCODE	+ SCHEMA.md, partial		

policy history, premiums, 6.8 M bookings · people, brokers, events · parameters & market data · dictionary

no source code

What must be rebuilt, exactly

- **t0** Commission of the broker on every premium
- **t1** Next year's due dates and collection dates
- **t2** Projected maturity value of the signed offer
- **t3** Full fund accounting and surrender value
- **t4** Annual statement of premiums and costs
- **t5** Values overwritten by hand, which follow no rule
- **t7** Tax payouts and annual tax: added during the event, with few examples

Why the data alone is hard to read

Implicit links
Tables are joined only by column names (`PH_BRKNR` → `BR_BRKNR`). Versioned tables need joins on the date a value was valid.

Legacy formats
Dates like `1250131` (= 2025-01-31), fixed-point amounts, codes hidden at a position of a 7-character tariff key.

Two systems
The COBOL era and Neuron NG compute some values differently, and parameters are valid on the application date, not today.

Human noise
Manual overrides and outliers; only some of them are listed in `NREXCP`. Schema and data disagree in places.

Zero tolerance
A value counts only if it is exact. An approximation, or a neural network, is useless to an auditor.

02 First iteration: hand everything to a coding agent

Our naive baseline handed a coding agent the export, the schema and the train targets, and let it iterate. It worked surprisingly well, solving every task with excellent accuracy, **except** where input–output tuples were missing (the tax task added during the event) or the rules were incoherent and obscure.

It does not scale, though. The agent rereads everything at every turn, costing tokens and hours, and what it learns stays in a transcript: each new target restarts the loop, and a real legacy system has thousands. Many insurers also cannot send policy data to a frontier API, for privacy and data sovereignty.

	Coding agent alone	Ergo
The model reads	the whole export and all the tuples	a short report: the joins, cluster representatives, the best tree, the residuals
The model is called	at every step	only when the trees need a feature that the data does not have yet
A new target	reruns the whole loop	a task definition (JSON) and the same seven steps
The result	code, with its reasoning in a transcript	code, plus a ledger of every scored candidate and a control-flow graph in plain language

03 Ergo: let the data narrow the search, let the model fill the gaps

Every target goes through the same seven steps: classic, deterministic algorithms, each depending only on the artifacts before it. The language model is one step of seven, and may only propose features in a small language that the harness checks and scores exactly.

1 Profile
Mines keys and inclusion dependencies across all tables: the join graph, temporal joins, the grain and filter of the target.
`Desbordante`

2 Cluster
Relates the target to every input (ratio, difference, days, months, constant). A row joins a cluster only if its center reproduces it exactly.
`density · 7 roundings`

3 Optimal trees
Finds the conditions that select each cluster: sparse trees of depth ≤ 3 and the smallest feature sets that reach the optimum.
`GOSDT objective · exact DP`

4 Candidate pool
Each leaf becomes a rule card. Leaves whose constants match a parameter table fold into one lookup.
`rule cards · lookups`

5 Language model
Only when a tree lacks a feature: proposes ≤ 3 in a DSL, e.g. `at(BR_TIER, PR_RCVDT)`, parsed but never executed.
`Claude Code · open weights`

6 Exact score
Every candidate, to the cent, on discovery data and a holdout (one policy in five), into an append-only ledger.
`Int128 decimals`

7 Iterate
Another cluster fits a residual → a condition is missing, loop. None fits → a manual correction for t5.
`structure vs noise`

✓ Freeze
Templates turn the winner into Python in `submission.py`, which must reproduce the ledger score.
`check.py · radon MI`

↻ Steps 3 → 7 repeat on the residuals until no structure is left; what remains is noise or a value typed by hand.

Worked example: t0 commission, 225,858 train rows

`1 profile`	Grain **NRPREM** (94.8 % coverage), filter `PR_STATE = P`, temporal join on the policy history.
`2 cluster`	`commission / PR_AMTRC` forms **6 clusters**, from 0.5 % to 4 %, plus **10 noise points**.
`3 tree`	0 errors with 6 leaves. Minimal features: **broker tier on the money date** and the **premium type**.
`4 pool`	The 6 rates are `NRCOMR.CR_RATE` keyed by `(CR_TIER, CR_PAYTP)`: one lookup that also covers rates never seen.
`5 model`	Not needed. The scorer settles rounding: half-up beats half-even on one row.
`6–7 score`	**225,848 / 225,858**. All 10 residuals are noise: 6 are in `NREXCP` and 4 are new, so they go to t5.

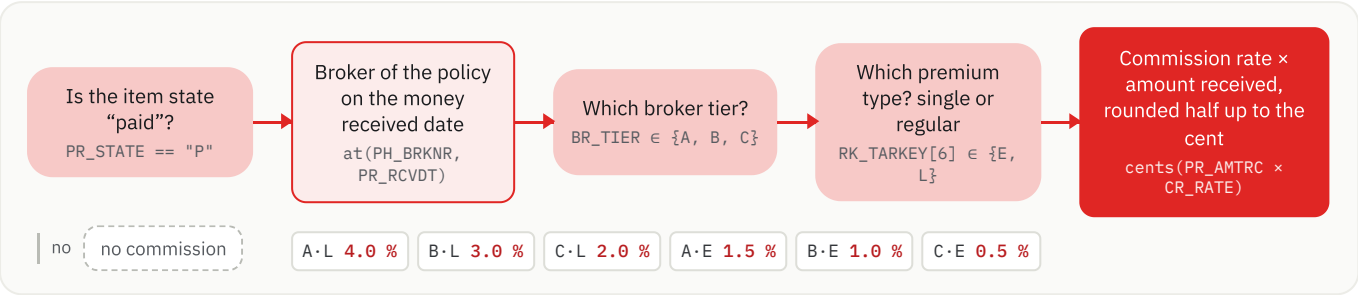
Any model, same guardrails

- **Claude Code, headless** (`claude -p`, no tools), for step 5 and an autonomous agent mode.
- **Open-weights models**: Qwen3.8-Flash-Next (125B MoE) on vLLM, or gpt-oss:20b on Ollama. Policy data stays on your infrastructure.
- **Sandboxed agents**: OpenCode with no network and no edits outside the solution.

Whatever the model proposes is a hypothesis. Only the exact scorer can promote it to a rule.

04 Auditable by design

Every frozen rule is also drawn as a **control-flow graph in plain language**, so a domain expert can validate it without reading Python. Every word comes from the export itself (`NRDICT`, `NRCODE`, `SCHEMA.md`), the code stays under each sentence, and any row can be replayed.



t0 commission as Ergo shows it: decisions in grey, the replayed path in red. The six leaves of the optimal tree are one lookup in `NRCOMR`, which is also the code that the freeze writes.

Hidden rules become rule cards that can be switched off

Each rule is one edit to the submission; ablation switches it off and shows which rows it makes right. Three that Neuron never documented:

`t3 · fx-day-before`

A premium is converted with the exchange rate of the day before the trade date.

switched off: the rate of the trade date

`t1 · good-friday`

Good Friday is a bank working day in Ticino: direct debits are collected.

switched off: a holiday, as in most cantons

`t2 · nearest-birthday`

Neuron NG takes the age at the nearest birthday, not at the last one.

switched off: the age at the last birthday

05 Results

Task		Accuracy, exact rows on the public policies	Ergo
t0	Commission PERFECT		**100.00 %** 80,592 / 80,592
t1	Next due dates PERFECT		**100.00 %** 5,005 / 5,005
t2	Projected maturity value PERFECT		**100.00 %** 986 / 986
t3	Surrender value PERFECT		**100.00 %** 3,028 / 3,028
t4	Annual statement PERFECT		**100.00 %** 11,648 / 11,648
t7	Tax payout		**90.00 %** 36 / 40
t7	Tax annual		**93.33 %** 28 / 30

Live leaderboard, submission #83, evaluated Sun 18 Oct 09:53: average accuracy 97.62 %, maintainability index 100.0. The t7 tax tasks were released during the event with only a few public samples.

06 Why it scales to a real migration

New targets, not new tools

A new target is a task definition made in a four-step wizard. The same seven steps run, with no task-specific code.

Cost follows the unknown

Cheap, deterministic algorithms do the bulk. The model sees a few representatives instead of millions of rows, and only when the trees are stuck.

Evidence an auditor accepts

Every candidate is in the ledger with its exact score, every prompt is stored, and every rule has a graph and an ablation. Noise is handed over case by case.